華中科技大學

课 程 实 验 报 告

课程名称： C 语言程序设计实验

专业班级：2502AI 大模型试验班

学　　号：U202517488

姓　　名：张书颢

指导教师：

报告日期：

软件学院

1　表达式和标准输入与输出实验

1.1 实验目的

（1）熟练掌握各种运算符的运算功能，操作数的类型，运算结果的类型及运算过程中的类型转换，重点是 C 语言特有的运算符，例如位运算符，问号运算符，逗号运算符等；熟记运算符的优先级和结合性。

（2）掌握 getchar, putchar, scanf 和 printf 函数的用法。

（3）掌握简单 C 程序（顺序结构程序）的编写方法。

1.2 实验内容

1 程序改错与跟踪调试

下面的实验 1-1 程序用来完成以下任务：

（1）输入华氏温度 f，将它转换成摄氏温度 c 后输出。

（2）输入圆的半径值 r，计算并输出圆的面积 s 。

（3）将 k 的高字节作为结果的低字节， p 的高字节作为结果的高字节，拼成一个新的整数后输出。

在这个程序中存在若干语法和逻辑错误，要求先编译程序改正语法错误，再采用单步执行的方式调试程序找出逻辑错误。在单步执行程序的过程中，观察以下变量值：

（1） 执行完 c = 5/9 * (f-32)，c 的值为多少？

（2） 执行完 scanf(“%f”, &r)，r 的值为多少？

（3） 执行完 newint = p&0xff00|k>>8， newint 的值是多少？表达式 k>>8 的值是多少？

根据观察结果分析代码并修改程序，使之能够正确完成指定任务。

```
#include<stdio.h>
#define PI 3.14159;
int main( void )
{
  int f ;
```

```
6     short p, k ;
7     double c , r , s ;
8   /*  任务  1 */
9   printf("Input Fahrenheit: " ) ;
10    scanf("%d", f ) ;
11    c = 5/9 * (f-32) ;
12    printf( " \n %d (F) = %.2f (C)\n\n ", f, c ) ;
13    /*  任务  2 */
14    printf("input the radius r:");
15    scanf("%f", &r);
16    s = PI * r * r;
17    printf("\nThe acreage is %.2f\n\n",&s);
18    /*  任务 3 */
19    k = 0xa1b2, p = 0x8432;
20    newint = p&0xff00|k>>8;
21    printf("new int = %#x\n\n",newint);
22    return 0;
23    }
```

解答：

错误修改：

1) 第 2 行的符号常量定义后不能有分号，应改为

   #define PI 3.14159

2) 第 5 行的 f 类型应该为 double，应改为

   double f ;

3) 第 10 行 f 的类型为 double，应改为

   scanf("%lf",&f);

4) 第 11 行 c 的类型为 double，应改为

   c = 1.0 * 5/9 * (f-32) ;

5) 第 12 行 c 的类型为 double，应改为

   printf("\n %.2f (F) = %.2f (C)\n\n ", f, c) ;

6) 第 15 行 r 的类型为 double，应改为

   scanf("%lf", &r);

7) 第 17 行存在 printf 语法错误，应改为

   printf("\nThe acreage is %.2f\n\n",s)

8）第 20 行存在 newint 类型、算法错误，应改为

int newint = (p&0xff00) | ((k&0xff00)>>8) ;

2 程序设计

（1）输入字符 c ，如果 c 是大写字母，则将 c 转换成对应的小写，否则 c 的值不变，输入 Ctrl+Z 程序结束。要求①用条件表达式；②字符的输入输出用 getchar 和 putchar 函数。程序应能循环接受用户的输入，直至输入 Ctrl+Z 程序结束。

解答：

1） 解题思路：算法流程如图 1-1 所示。

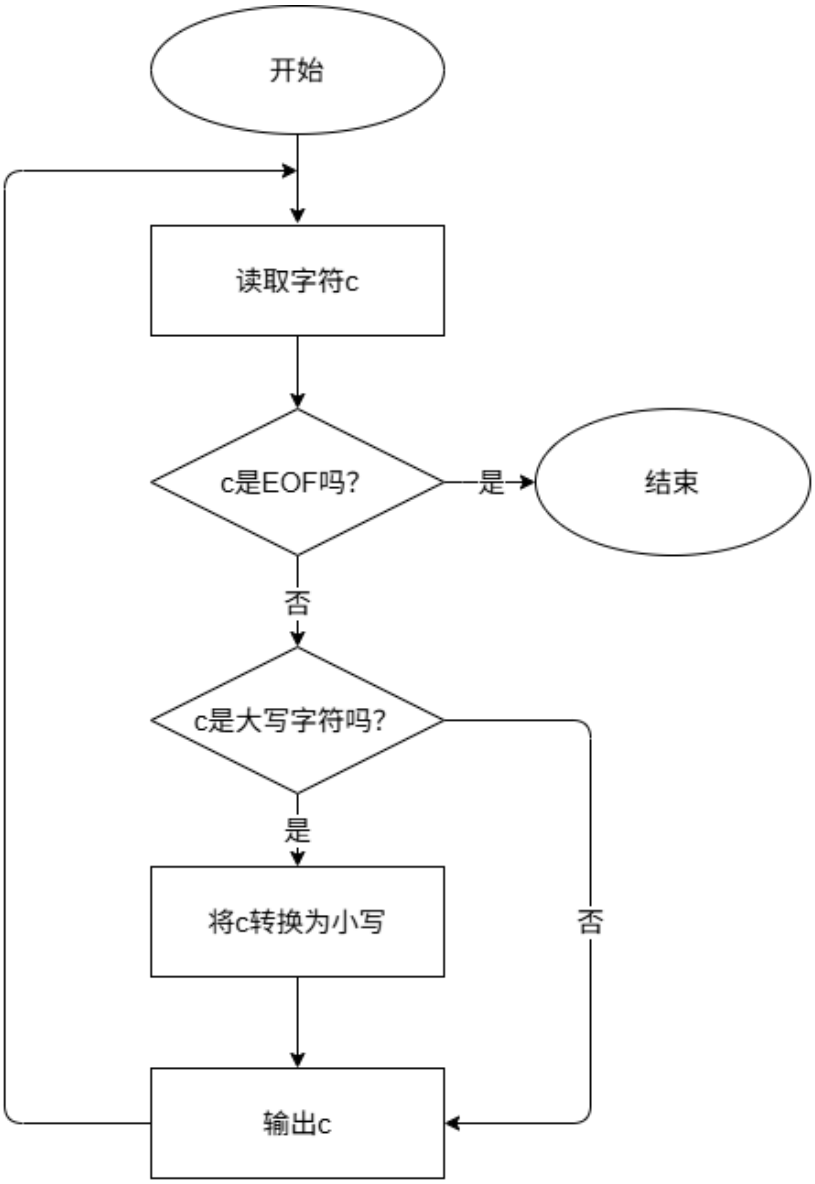

图 1-1 编程题 1 的程序流程图

2）源程序清单

```
#include <stdio.h>
int main(void) {
    int c;
    while ((c = getchar()) != EOF) {
        c = (c >= 'A' && c <= 'Z') ? (c + 'a' - 'A') : c;
        putchar(c);
    }
    return 0;
}
```

3）测试

（a） 测试数据：

表 1-1 编程题 1 的测试数据

测试用例	程序输入	理论结果	实际结果
用例 1	a	a	a
用例 2	B	b	b
用例 3	,	,	,

（b） 对应测试数据的运行结果如图 1-2 所示

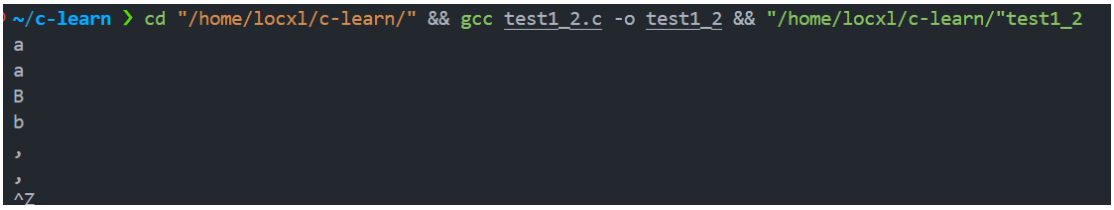

图 1-2 编程题 1 的测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（2）输入无符号短整数 x，m，n（0 ≤m≤ 15，1 ≤ n≤ 16-m），取出 x 从第m位开始向左的 n 位（m从右至左编号为 0～15），并使其向左端（第 15 位）靠齐。要求：①检查 m 和 n 的范围；②x 的值以十六进制输入，m 和 n 以十进制输入；③结果以十六进制输出。

解答：

1) 解题思路：算法流程如图 1-3 所示。

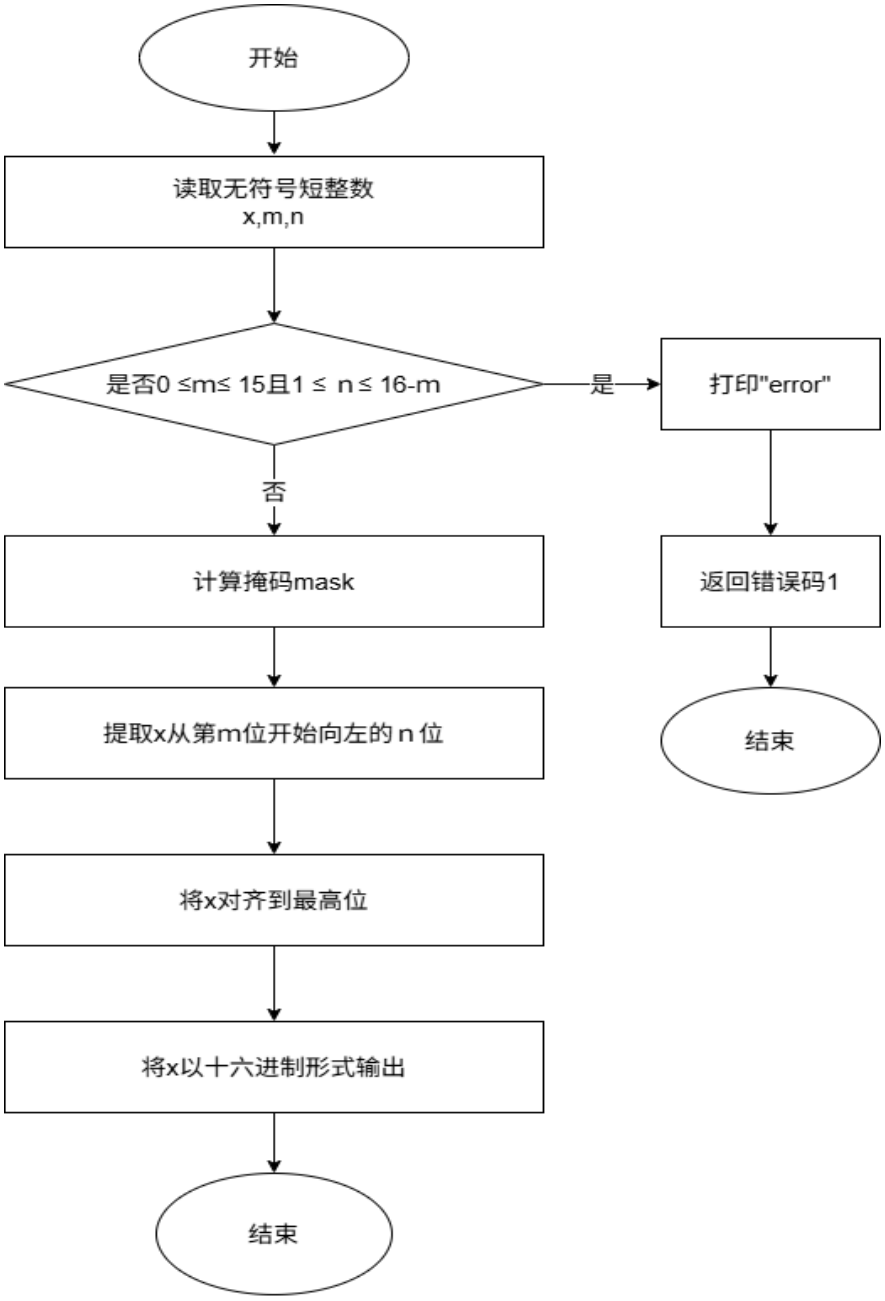

图 1-3 编程题 2 的程序流程图

2）程序清单

```
#include <stdio.h>

int main() {
    unsigned short x, m, n;
    scanf("%hx %hu %hu", &x, &m, &n);
    if (m < 0 || m > 15 || n < 1 || n > 16 - m) {
        printf("error\n");
        return 1;
```

```
    }
    unsigned short mask = ((1 << n) - 1) << m;
    unsigned short result = (x & mask) >> m;
    result <<= (16 - n);
    printf("%04hx\n", result);
    return 0;
}
```

3）测试

（a） 测试数据：

如表 1-2 所示。

表 1-2 编程题 2 的测试数据

测试用例	程 序 输 入			理 论 结 果	运 行 结 果
	X	m	N		
用例 1	0100 0110 1000 0000（4680）	7	4	计算结果 1101 0000 0000 0000　即 D000	D000
用例 2	1101 0101 1000 0011（D583）	16	1	error（m 值超范围）	error
用例 3	1101 0101 1000 0011（D583）	13	5	error（n 值超范围）	error

（b） 对应测试测试用例 1 的运行结果如图 1-4 所示。

```
● ~/c-learn ❯ cd "/home/locxl/c-learn/" && gcc test1_3.c -o test1_3 && "/home/locxl/c-learn/"test1_3
  4680 7 4
  d000
```

图 1-4 编程题 2 的测试用例一的运行结果

对应测试测试用例 2 的运行结果如图 1-5 所示。

```
~/c-learn ❯ cd "/home/locxl/c-learn/" && gcc test1_3.c -o test1_3 && "/home/locxl/c-learn/"test1_3
d583 16 1
error
```

图 1-5 编程题 2 的测试用例二的运行结果

对应测试测试用例 3 的运行结果如图 1-6 所示。

```
~/c-learn ❯ cd "/home/locxl/c-learn/" && gcc test1_3.c -o test1_3 && "/home/locxl/c-learn/"test1_3
d583 13 5
error
```

图 1-6 编程题 2 的测试用例三的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（3）IP 地址通常是 4 个用句点分隔的小整数（即点分十进制），但这些地址在机器中是用一个无符号长整型数表示的。例如 3232235876，其机内二进制表示就是 11000000 10101000 00000001 01100100，按照 8 位一组用点分开，该 IP 地址就写成 192.168.1.100。

读入无符号长整型数表示的互联网 IP 地址，对其译码，以常见的点分十进制形式输出。要求循环输入和输出，直至输入 Ctrl+Z 结束。

解答：

1） 解题思路：算法流程如图 1-7 所示。

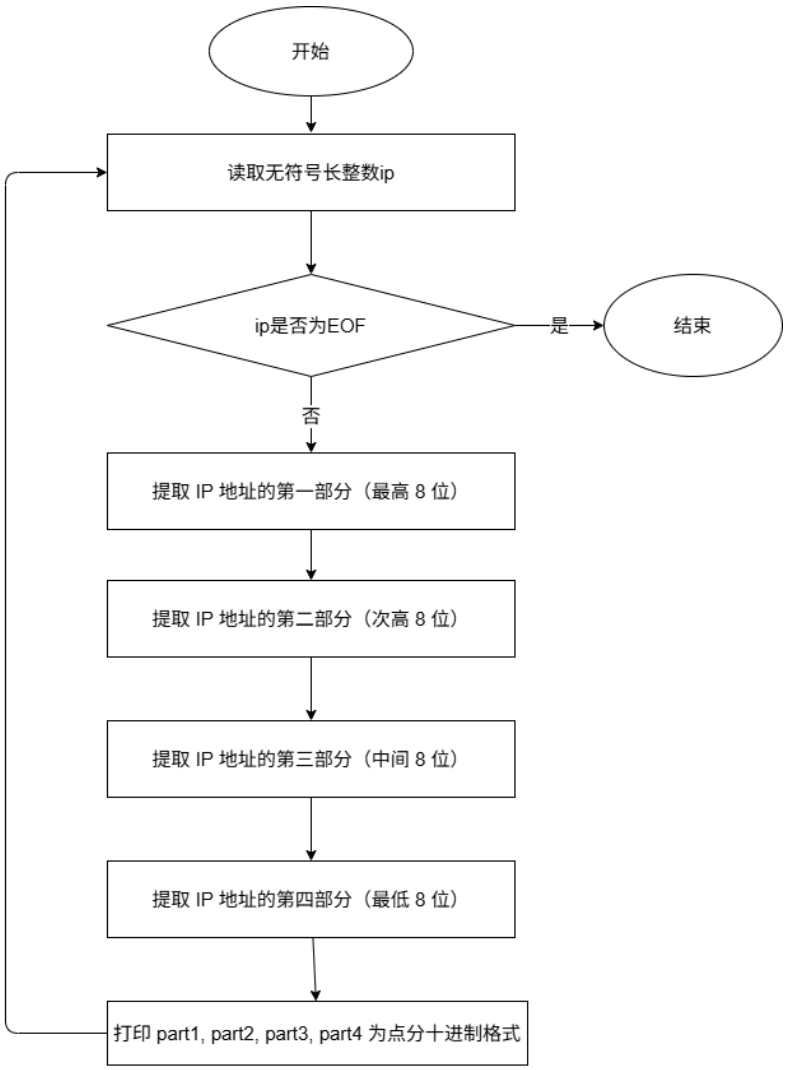

图 1-7 编程题 3 的程序流程图

2）程序清单

```
#include <stdio.h>

int main() {
    unsigned long ip;

    // 循环读取输入，直到文件结束（EOF）
    while (scanf("%lu", &ip) != EOF) {
        // 通过位运算将 32 位整数拆分为 4 个 8 位部分，对应 IP 地址的 4 段
        int part1 = (ip >> 24) & 0xFF; // 取最高 8 位
        int part2 = (ip >> 16) & 0xFF; // 取次高 8 位
        int part3 = (ip >> 8) & 0xFF;   // 取次低 8 位
        int part4 = ip & 0xFF;              // 取最低 8 位

        // 按点分十进制格式输出 IP 地址
        printf("%d.%d.%d.%d\n", part1, part2, part3, part4);
    }

    return 0;
}
```

3）测试

（a） 测试数据：

如表 1-3 所示。

表 1-3 编程题 3 的测试数据

测试用例	程 序 输 入	理 论 结 果	运 行 结 果
用例 1	3232235777	192.168.1.1	192.168.1.1
用例 2	0	0.0.0.0	0.0.0.0
用例 3	4294967295	255.255.255.255	255.255.255.255
用例 4	167772161	10.0.0.1	10.0.0.1

（b） 对应测试测试用例的运行结果如图 1-8 所示。

```
~/c-learn ❯ cd "/home/locxl/c-learn/" && gcc test1_4.c -o test1_4 && "/home/locxl/c-learn/"test1_4
3232235777
192.168.1.1
0
0.0.0.0
4294967295
255.255.255.255
167772161
10.0.0.1
^Z
```

图 1-8 编程题 3 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

1.3 实验小结

在本次实验中，我重点对程序中的逻辑错误进行了调试和修正。例如，在华氏温度转摄氏温度的计算中，初始代码使用整数除法导致结果为 0，通过将其调整为浮点运算（1.0 * 5 / 9）解决了问题。在位运算拼接整数的任务中，逻辑错误导致结果不符合预期，经过分析发现需要先提取 p 的高字节和 k 的低字节，再进行拼接，最终成功完成了任务。

此外，我还学会了如何通过范围检查避免数值越界等问题，以及如何结合单步调试观察变量变化来定位问题。这次实验让我深刻认识到编程细节的重要性，同时也强化了我对 C 语言运算符和输入输出函数的理解。总体而言，理论与实践的结合使我进一步掌握了程序设计的基本技能。